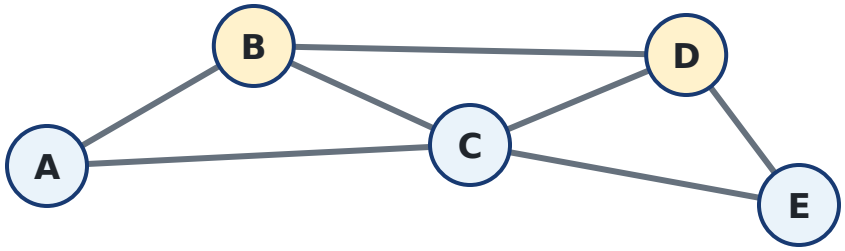


Aplicações Computacionais de Grafos com Java

Da modelagem do problema à escolha do algoritmo



Prof. Calvetti

Engenharia da Computação



Objetivos da aula

- Reconhecer quando um problema cotidiano pode ser representado por um grafo.
- Definir vértices, arestas, direção e pesos de acordo com a pergunta do problema.
- Escolher uma representação e um algoritmo compatíveis com as restrições existentes.
- Interpretar, adaptar e testar implementações em Java.
- Justificar a solução por correção, custo computacional e limites de aplicação.

Competência central: transformar uma situação real em uma decisão computacional justificável.



Organização da aula

Parte	Conteúdo
1	Modelagem e escolha do algoritmo
2	BFS e propagação de alertas
3	Dijkstra e rotas de menor custo
4	Dependências e ordenação topológica
5	Emparelhamento, Euler e Hamilton
6	Exercícios e síntese

Conhecimentos mobilizados

Estruturas e propriedades

- Grafo direcionado e não direcionado.
- Grafo ponderado e não ponderado.
- Lista e matriz de adjacência.
- Caminhos, ciclos e conectividade.
- Grafos bipartidos, eulerianos e hamiltonianos.

Algoritmos e técnicas

- Busca em largura e profundidade.
- Fila e fila de prioridade.
- Algoritmos gulosos.
- Busca de caminhos aumentantes.
- Análise de complexidade.



01

Modelagem orientada ao problema

O grafo depende da pergunta que precisa ser respondida



Por que grafos aparecem no cotidiano

Situação	Vértices	Arestas
Mobilidade	Locais	Trechos
Comunicação	Pessoas ou roteadores	Canais
Dependências	Tarefas ou módulos	Pré-requisitos
Alocação	Recursos e demandas	Compatibilidades
Infraestrutura	Pontos físicos	Conexões



Cinco perguntas de modelagem

1. Quais entidades se tornam vértices?
2. O que caracteriza uma conexão?
3. A conexão possui direção?
4. Existe custo, tempo, distância ou capacidade?
5. A solução procura alcançar, ordenar, cobrir ou atribuir?

A mesma cidade pode gerar grafos diferentes

Pergunta sobre deslocamento

- Vértices: cruzamentos.
- Arestas: trechos viários.
- Peso: tempo de viagem.
- Objetivo: menor custo acumulado.

Pergunta sobre atendimento

- Vértices: equipes e chamados.
- Arestas: compatibilidades.
- Estrutura: grafo bipartido.
- Objetivo: maximizar atribuições.

A realidade não determina sozinha o grafo. A pergunta define a abstração útil.



Problema, estrutura e algoritmo

Necessidade	Estrutura	Técnica
Alcançar um destino	Grafo sem pesos	BFS ou DFS
Menor número de conexões	Grafo sem pesos	BFS
Menor custo acumulado	Pesos não negativos	Dijkstra
Ordenar dependências	Digrafo acíclico	Ordenação topológica
Atribuir recursos	Grafo bipartido	Emparelhamento
Cobrir conexões	Graus dos vértices	Trilha de Euler



Lista ou matriz de adjacência

Critério	Lista	Matriz
Memória	O(V + E)	$O(V^2)$
Enumerar vizinhos	O(grau(v))	O(V)
Consultar uma aresta	Depende do grau	O(1)
Perfil adequado	Grafo esparso	Grafo pequeno ou denso

A representação também faz parte da solução algorítmica.



Modelo-base em Java

```
class Aresta {
    final int destino;
    final int peso;

    Aresta(int destino, int peso) {
        this.destino = destino;
        this.peso = peso;
    }
}

class Grafo {
    private final List<List<Aresta>> adj;

    Grafo(int vertices) {
        adj = new ArrayList<>();
        for (int i = 0; i < vertices; i++)
            adj.add(new ArrayList<>());
    }
}
```



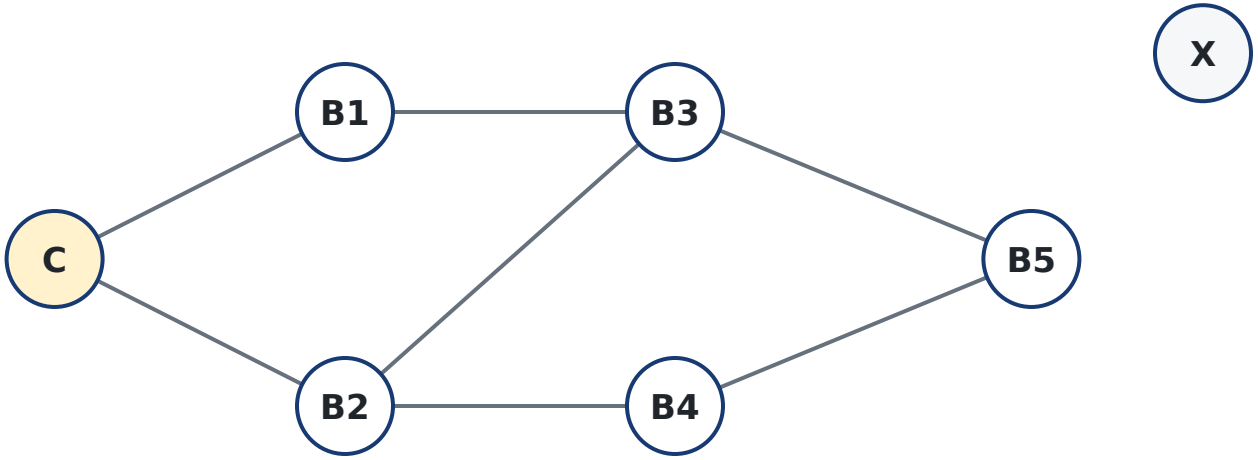
02

Aplicação 1

Propagação de alertas e menor número de conexões com BFS

Rede municipal de alertas

Uma central precisa saber quais bairros receberão um alerta e quantas retransmissões serão necessárias.



BFS calcula a distância em número de arestas e identifica o vértice isolado X.



BFS aplicada ao problema

Estado mantido

- Fila de vértices descobertos.
- dist[v] = menor quantidade de arestas.
- anterior[v] = predecessor na árvore BFS.
- Valor -1 indica vértice ainda não alcançado.

Processamento

- Retirar o primeiro vértice da fila.
- Examinar seus vizinhos.
- Descobrir apenas vizinhos não visitados.
- Registrar distância e predecessor.



Rastreamento da BFS

Etapa	Fila antes	Vértice	Novos vértices
1	[C]	C	B1, B2
2	[B1, B2]	B1	B3
3	[B2, B3]	B2	B4
4	[B3, B4]	B3	B5
5	[B4, B5]	B4	nenhum
6	[B5]	B5	nenhum



BFS em Java: inicialização

```
static ResultadoBFS bfs(List<List<Integer>> adj, int origem) {
    int n = adj.size();
    int[] dist = new int[n];
    int[] anterior = new int[n];

    Arrays.fill(dist, -1);
    Arrays.fill(anterior, -1);

    Queue<Integer> fila = new ArrayDeque<>();
    dist[origem] = 0;
    fila.add(origem);

    // processamento no próximo slide
```

O valor -1 permite distinguir um vértice não visitado de uma distância válida.



BFS em Java: processamento

```
    while (!fila.isEmpty()) {
        int u = fila.remove();

        for (int v : adj.get(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                anterior[v] = u;
                fila.add(v);
            }
        }
    }

    return new ResultadoBFS(dist, anterior);
}
```

Complexidade com lista de adjacência: O(V + E).



Reconstrução do caminho

```
static List<Integer> caminho(int destino, int[]
anterior) {
    List<Integer> rota = new ArrayList<>();

    for (int v = destino; v != -1; v = anterior[v])
        rota.add(v);

    Collections.reverse(rota);
    return rota;
}
```

Exemplo: C → B1 → B3 → B5, com 3 retransmissões.



Quando BFS resolve o problema

Adequado

- Alcançabilidade.
- Menor número de arestas.
- Todos os deslocamentos têm o mesmo custo lógico.
- Exploração por níveis.

Inadequado

- Arestas possuem custos diferentes.
- O objetivo minimiza distância, tempo ou preço.
- Existem restrições que mudam durante a busca.
- O estado não depende apenas do vértice.



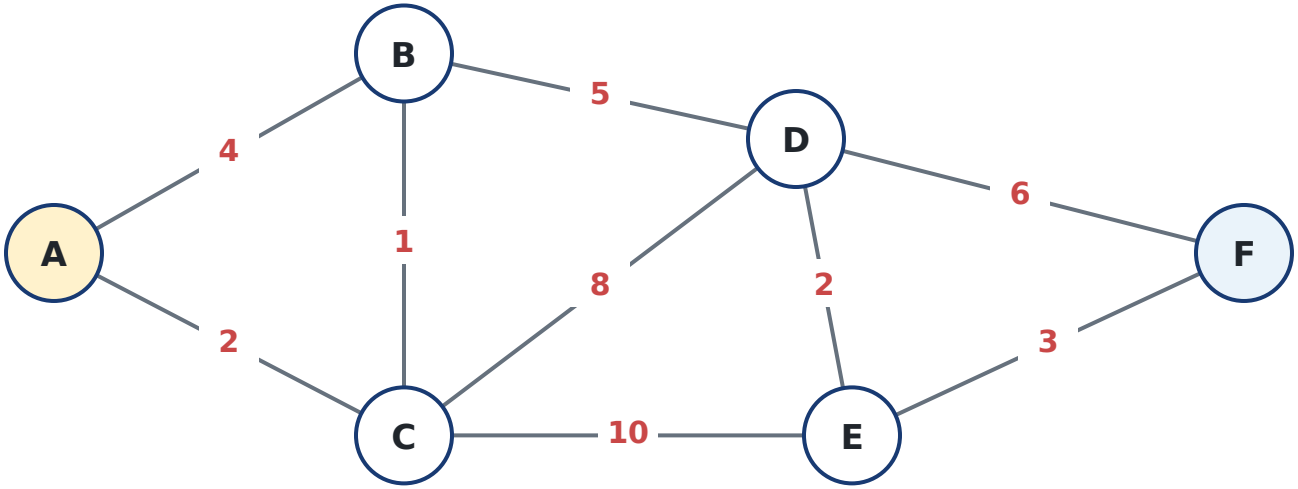
03

Aplicação 2

Rotas de menor custo com Dijkstra

Rede de mobilidade ponderada

Os pesos representam minutos de deslocamento. A meta é minimizar a soma dos pesos.



Qual é a rota de menor tempo entre A e F?



Ideia central de Dijkstra

• A origem começa com distância zero. Os demais vértices começam com infinito.

• A fila de prioridade seleciona a menor distância provisória.

• O relaxamento testa se o caminho por u melhora a distância de v.

• Quando a melhor entrada de u sai da fila, suas arestas podem ser processadas.

• A correção exige pesos não negativos.

Relaxamento: se dist[u] + peso(u,v) < dist[v], então atualize dist[v] e anterior[v].



Dijkstra: primeiras relaxações

Retirado	A	B	C	D	E	F
Início	0	∞	∞	∞	∞	∞
A	0	4	2	∞	∞	∞
C	0	3	2	10	12	∞
B	0	3	2	8	12	∞

Ao processar C, o caminho A-C-B custa 3 e melhora o valor anterior 4.

Dijkstra: conclusão

Retirado	A	B	C	D	E	F
D	0	3	2	8	10	14
E	0	3	2	8	10	13
F	0	3	2	8	10	13

Rota mínima: A → C → B → D → E → F

Custo total: 2 + 1 + 5 + 2 + 3 = 13 minutos



Fila de prioridade em Java

```
class NoDistancia implements Comparable<NoDistancia> {
    final int vertice;
    final int distancia;

    NoDistancia(int vertice, int distancia) {
        this.vertice = vertice;
        this.distancia = distancia;
    }

    @Override
    public int compareTo(NoDistancia outro) {
        return Integer.compare(this.distancia,
outro.distancia);
    }
}
```



Dijkstra em Java: inicialização

```
static Resultado dijkstra(List<List<Aresta>> adj, int origem) {
    int n = adj.size();
    int[] dist = new int[n];
    int[] anterior = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    Arrays.fill(anterior, -1);

    PriorityQueue<NoDistancia> fila = new PriorityQueue<>();
    dist[origem] = 0;
    fila.add(new NoDistancia(origem, 0));

    // relaxamentos no próximo slide
```



Dijkstra em Java: relaxamento

```
while (!fila.isEmpty()) {
    NoDistancia atual = fila.remove();
    int u = atual.vertice;

    if (atual.distancia != dist[u]) continue;

    for (Aresta a : adj.get(u)) {
        int nova = dist[u] + a.peso;
        if (nova < dist[a.destino]) {
            dist[a.destino] = nova;
            anterior[a.destino] = u;
            fila.add(new NoDistancia(a.destino, nova));
        }
    }
}
return new Resultado(dist, anterior);
```

Interpretação do resultado

O algoritmo entrega

- Custo mínimo da origem para cada vértice.
- Predecessor escolhido em cada melhoria.
- Árvore de menores caminhos.
- Rota reconstruída do destino até a origem.

A aplicação ainda decide

- Como obter e atualizar os pesos.
- Como tratar vértices inalcançáveis.
- Quando recalcular as rotas.
- Como apresentar alternativas ao usuário.



Limites de Dijkstra

• Pesos negativos invalidam a escolha gulosa usada pelo algoritmo.

• Mudanças de peso após o cálculo podem tornar a rota obsoleta.

• Um único peso pode não representar tempo, custo, risco e acessibilidade simultaneamente.

• Vértices e arestas precisam representar corretamente as restrições do domínio.

• O menor custo matemático pode não corresponder à melhor decisão humana.

Um algoritmo correto sobre um modelo inadequado produz uma resposta inadequada.



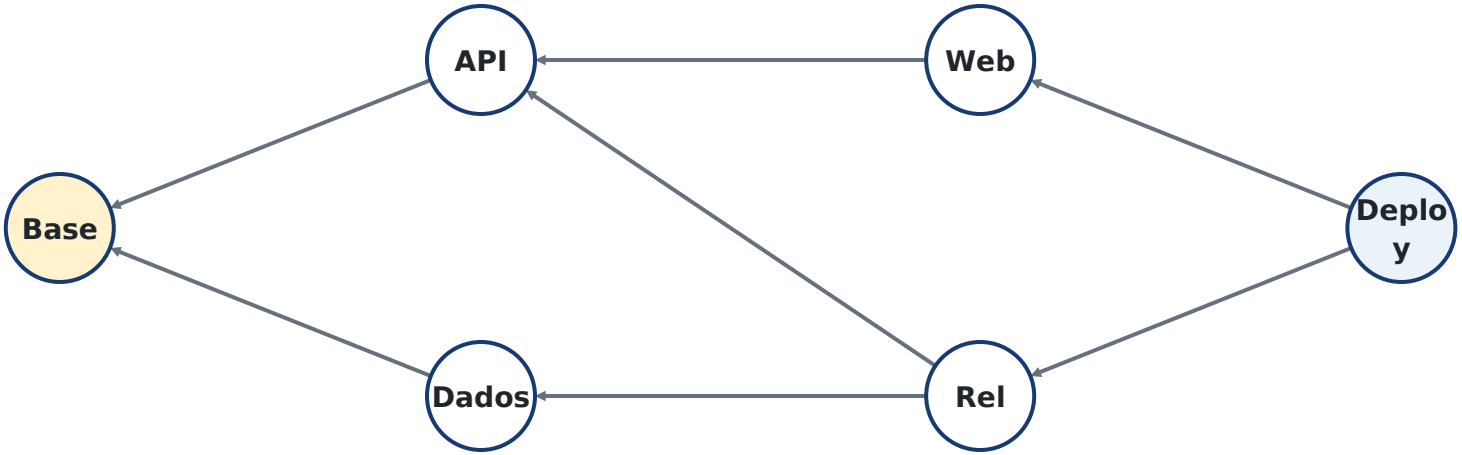
04

Aplicação 3

Dependências e ordenação topológica

Dependências entre módulos

Uma aresta A → B indica que B depende da conclusão de A.



O grafo precisa ser acíclico para admitir uma ordem topológica.



Algoritmo de Kahn

- Calcular o grau de entrada de cada vértice.
- Inserir na fila os vértices com grau de entrada zero.
- Retirar um vértice, adicioná-lo à ordem e remover logicamente suas arestas.
- Quando o grau de entrada de um vizinho chegar a zero, inseri-lo na fila.
- Se a ordem contiver menos de V vértices, existe uma dependência circular.

Diferentes escolhas na fila podem produzir diferentes ordens válidas.



Ordenação topológica: rastreamento

Passo	Disponíveis	Escolhido	Novos disponíveis
1	Base	Base	API, Dados
2	API, Dados	API	Web
3	Dados, Web	Dados	nenhum
4	Web	Web	Rel
5	Rel	Rel	Deploy
6	Deploy	Deploy	nenhum



Ordenação topológica em Java: preparação

```
static List<Integer> ordemTopologica(List<List<Integer>> adj) {
    int n = adj.size();
    int[] entrada = new int[n];

    for (int u = 0; u < n; u++)
        for (int v : adj.get(u))
            entrada[v]++;

    Queue<Integer> fila = new ArrayDeque<>();
    for (int v = 0; v < n; v++)
        if (entrada[v] == 0) fila.add(v);

    List<Integer> ordem = new ArrayList<>();
```



Ordenação topológica em Java: processamento

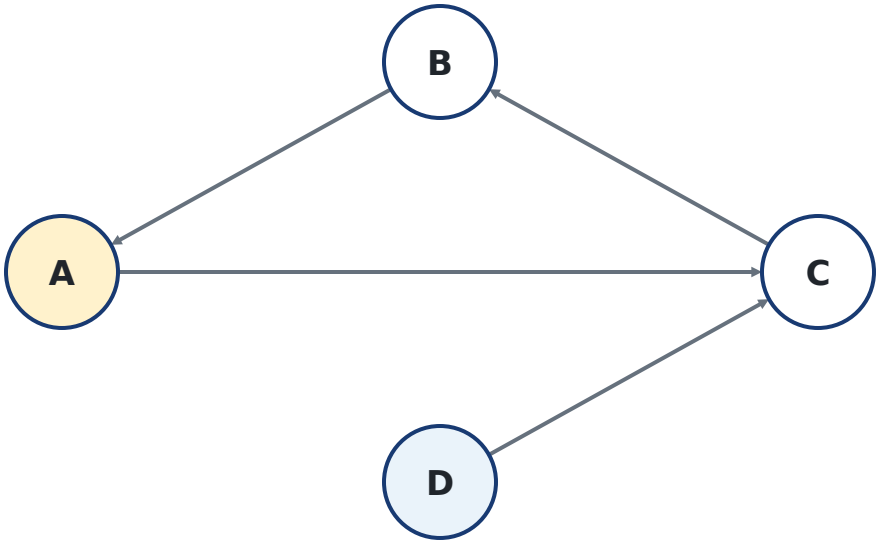
```
    while (!fila.isEmpty()) {
        int u = fila.remove();
        ordem.add(u);

        for (int v : adj.get(u)) {
            entrada[v]--;
            if (entrada[v] == 0) fila.add(v);
        }
    }

    if (ordem.size() != n)
        throw new IllegalStateException("Dependência
circular");

    return ordem;
}
```

Diagnóstico de uma dependência circular



- Nenhum vértice do ciclo A-B-C alcança grau de entrada zero.
- O algoritmo encerra antes de processar todos os vértices.
- D também permanece bloqueado porque depende de C.

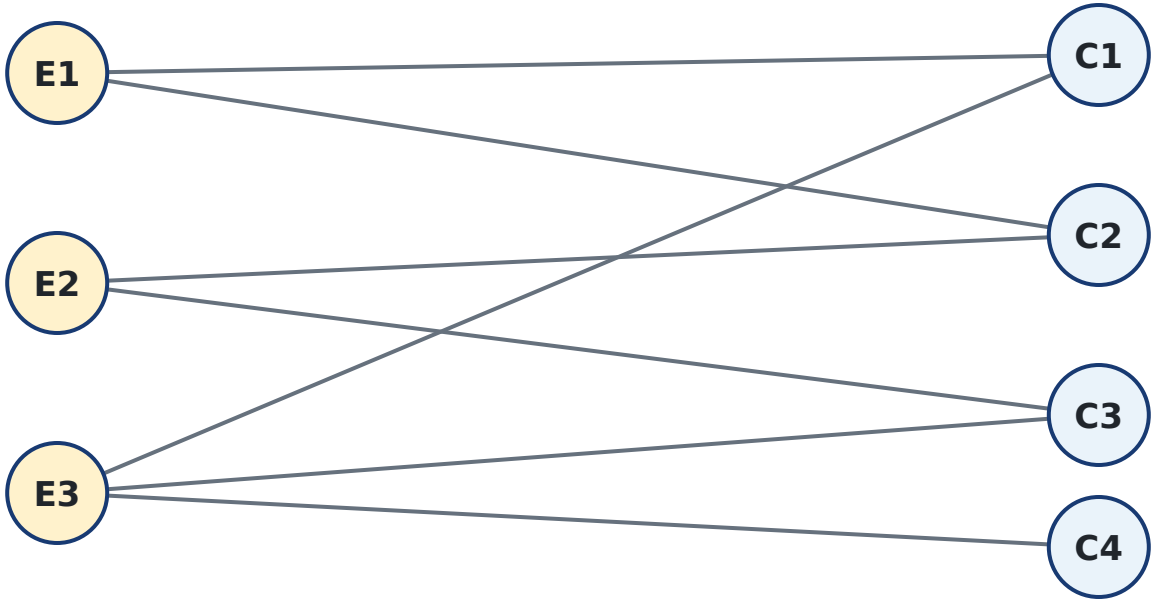
05

Aplicação 4

Atribuição de recursos por emparelhamento

Equipes e chamados técnicos

Uma aresta indica que a equipe possui competência para atender o chamado.



Meta: maximizar a quantidade de pares sem repetir equipe nem chamado.



Caminho aumentante

- A busca parte de uma equipe ainda não atribuída.
- Uma aresta livre pode criar uma nova atribuição.
- Uma aresta já usada pode ser desfeita se outra atribuição preservar o atendimento anterior.
- A alternância entre arestas livres e usadas aumenta o emparelhamento em uma unidade.
- A ausência de caminho aumentante caracteriza um emparelhamento máximo.

Máximo não significa necessariamente perfeito: pode haver mais chamados do que equipes.



Emparelhamento em Java: busca de atribuição

```
static boolean tenta(int equipe,
        List<List<Integer>> adj,
        boolean[] visitado,
        int[] equipeDoChamado) {

    for (int chamado : adj.get(equipe)) {
        if (visitado[chamado]) continue;
        visitado[chamado] = true;

        if (equipeDoChamado[chamado] == -1 ||
            tenta(equipeDoChamado[chamado], adj,
                  visitado, equipeDoChamado)) {
            equipeDoChamado[chamado] = equipe;
            return true;
        }
    }
    return false;
}
```



Emparelhamento em Java: solução máxima

```
static int emparelhamentoMaximo(
        List<List<Integer>> adj, int chamados) {

    int[] equipeDoChamado = new int[chamados];
    Arrays.fill(equipeDoChamado, -1);
    int total = 0;

    for (int equipe = 0; equipe < adj.size(); equipe++) {
        boolean[] visitado = new boolean[chamados];
        if (tenta(equipe, adj, visitado, equipeDoChamado))
            total++;
    }
    return total;
}
```

Implementação didática: O(VE) no pior caso.

Cobertura de ruas: problema euleriano

Pergunta operacional

- É possível percorrer cada rua exatamente uma vez?
- O interesse está nas arestas.
- Aplicações: inspeção, varrição e leitura de medidores.

Critério estrutural

- Todos os graus pares: circuito euleriano.
- Exatamente dois graus ímpares: trilha aberta.
- Mais de dois graus ímpares: nenhuma trilha euleriana.

Na prática, ruas adicionais podem precisar ser repetidas para tornar a rota executável.

Visita a depósitos: problema hamiltoniano

Pergunta operacional

- É possível visitar cada depósito uma única vez?
- O interesse está nos vértices.
- A rota pode exigir retorno ao ponto inicial.

Implicações computacionais

- O critério de graus de Euler não se aplica.
- A busca exata pode crescer exponencialmente.
- Problemas grandes recorrem a limites e heurísticas.

Encontrar uma rota válida e encontrar a rota ótima são problemas diferentes.



Euler e Hamilton: escolha pelo objeto visitado

Questão	Euler	Hamilton
O que deve ser coberto?	Arestas	Vértices
Pode repetir vértice?	Sim	Não, salvo o fechamento
Pode repetir aresta?	Não	A rota usa arestas disponíveis
Critério simples?	Graus dos vértices	Não existe critério geral equivalente
Exemplo	Inspecionar ruas	Visitar depósitos



Escolha da representação no projeto

Cenário	Escolha	Justificativa
Rede viária ampla	Lista	Poucas conexões por local
Mapa pequeno e denso	Matriz	Consulta direta de aresta
Dependências	Lista	Percorrer sucessores
Compatibilidades	Lista bipartida	Arestas esparsas
Custos entre poucas unidades	Matriz ponderada	Acesso O(1)



Arquitetura orientada a objetos

Classe	Responsabilidade
Vertice	Identidade e dados do domínio
Aresta	Destino, peso e propriedades da conexão
Grafo	Representação e operações estruturais
AlgoritmoBFS	Alcançabilidade, distâncias e predecessores
AlgoritmoDijkstra	Menores custos e reconstrução de rotas
Aplicacao	Entrada, validação e apresentação dos resultados

Complexidade das soluções

Solução	Estrutura principal	Complexidade
BFS	Fila	O(V + E)
Dijkstra	Fila de prioridade	O((V + E) log V)
Ordenação topológica	Fila e graus de entrada	O(V + E)
Emparelhamento didático	DFS por equipe	O(VE)
Hamilton por força bruta	Permutações	Crescimento fatorial

A complexidade precisa ser interpretada junto com o tamanho e a densidade do grafo.



Roteiro de decisão

1 **Formalizar a pergunta**

2 **Definir V e E**

3 **Decidir direção e pesos**

4 **Escolher a representação**

5 **Selecionar o algoritmo**

6 **Validar limites e casos extremos**

Depois de implementar: testar grafo vazio, vértice isolado, destino inalcançável e entradas inválidas.

06

Lista de exercícios

Dez situações para modelar, decidir e justificar

Exercícios 1 e 2

1 — Rede de contatos

- Contatos: A-B, A-C, B-D, C-E, E-F.
- Modele o grafo e liste quem recebe uma informação iniciada por A.
- Informe a distância de A até F.
- Justifique BFS ou DFS.

2 — Conexões aéreas

- Voos: SP-RJ, SP-BH, RJ-SSA, BH-BSB, BSB-REC, SSA-REC.
- Encontre o menor número de voos entre SP e REC.
- Indique uma rota mínima.
- Explique o papel dos predecessores.

Exercícios 3 e 4

3 — Rota de ambulância

- Pesos: A-B=4, A-C=2, C-B=1, B-D=5, C-D=8, D-E=2, E-F=3, D-F=6.
- Aplique Dijkstra de A até F.
- Mostre distâncias e predecessores.
- Reconstrua a rota.

4 — Escolha do algoritmo

- Caso I: todas as ligações valem 1.
- Caso II: pesos positivos distintos.
- Caso III: uma aresta possui peso -3.
- Escolha BFS, Dijkstra ou nenhum dos dois e justifique.

Exercícios 5 e 6

5 — Ordem de instalação

- A→C, B→C, C→D, B→E, D→F, E→F.
- Calcule os graus de entrada.
- Produza uma ordem topológica válida.
- Existe apenas uma resposta?

6 — Dependência circular

- A→B, B→C, C→A e C→D.
- Execute conceitualmente o algoritmo de Kahn.
- Explique por que ele não conclui.
- Identifique os módulos afetados.

Exercícios 7 e 8

7 — Técnicos e chamados

• T1:{C1,C2}; T2:{C2,C3}; T3:{C1,C3,C4}.

• Encontre um emparelhamento máximo.

• É possível atender os quatro chamados simultaneamente?

• Justifique pela estrutura do problema.

8 — Coleta em ruas

• Graus dos cruzamentos: 4, 2, 3, 3, 2, 2.

• Classifique a rota como circuito, trilha aberta ou impossível.

• Indique onde a rota deve começar e terminar.

• Explique o critério utilizado.

Exercícios 9 e 10

9 — Depósitos de entrega

• A empresa quer visitar cada depósito uma única vez e retornar à origem.

• O problema é euleriano ou hamiltoniano?

• O grau par de todos os vértices garante a solução?

• Explique a diferença entre existência e otimização.

10 — Projeto em Java

• Uma universidade quer orientar alunos entre prédios considerando tempo e acessibilidade.

• Defina V, E, direção e pesos.

• Escolha representação e algoritmo.

• Proponha classes, testes e casos extremos.

Gabaritos 1 e 2

1 — Rede de contatos

- Alcançáveis: A, B, C, D, E e F.
- Uma árvore BFS possível: A-B, A-C, B-D, C-E, E-F.
- dist(A,F)=3, pelo caminho A-C-E-F.
- BFS é necessária para garantir a menor quantidade de conexões.

2 — Conexões aéreas

- Menor quantidade: 3 voos.
- Rota possível: SP-RJ-SSA-REC.
- Outra rota mínima: SP-BH-BSB-REC.
- Os predecessores permitem reconstruir a rota do destino até a origem.

Gabaritos 3 e 4

3 — Rota de ambulância

• Distâncias finais: A=0, C=2, B=3, D=8, E=10, F=13.

• Predecessores: C←A, B←C, D←B, E←D, F←E.

• Rota: A-C-B-D-E-F.

• Custo total: 13.

4 — Escolha do algoritmo

• Caso I: BFS, pois cada aresta representa o mesmo custo.

• Caso II: Dijkstra, pois os pesos são distintos e não negativos.

• Caso III: nenhum dos dois é apropriado para garantir correção.

• A escolha decorre das propriedades dos pesos.

Gabaritos 5 e 6

5 — Ordem de instalação

• Entradas: A=0, B=0, C=2, D=1, E=1, F=2.

• Ordem possível: A, B, C, D, E, F.

• Também é possível iniciar por B e permutar tarefas independentes.

• A ordem não é única.

6 — Dependência circular

• A, B e C possuem dependências que formam um ciclo.

• Nenhum deles fica disponível com grau de entrada zero.

• D também não é processado porque depende de C.

• ordem.size() < V confirma a presença de ciclo.

Gabaritos 7 e 8

7 — Técnicos e chamados

• Emparelhamento máximo possível: T1-C2, T2-C3, T3-C4.

• O tamanho máximo é 3.

• Quatro chamados não podem ser atendidos por apenas três técnicos.

• Um emparelhamento máximo não precisa ser perfeito no lado dos chamados.

8 — Coleta em ruas

• Existem exatamente dois vértices de grau ímpar: os de grau 3.

• Existe trilha euleriana aberta, se o grafo for conectado.

• A rota começa em um vértice ímpar e termina no outro.

• Não existe circuito euleriano.

Gabaritos 9 e 10

9 — Depósitos de entrega

• O problema é hamiltoniano porque exige visitar vértices.

• Graus pares garantem condição euleriana, não hamiltoniana.

• Uma rota existente ainda pode não ter o menor custo.

• Otimizar exige considerar os pesos das arestas.

10 — Projeto em Java

• Vértices: prédios; arestas: trajetos acessíveis.

• Pesos: tempo, com filtro prévio de acessibilidade.

• Lista de adjacência e Dijkstra com PriorityQueue.

• Classes: Predio, Trajeto, Grafo, Roteador e Aplicacao.

• Testes: origem=destino, prédio isolado, rota ausente e pesos inválidos.

Síntese e recuperação ativa

- Qual propriedade do problema determina se uma aresta deve ter direção?
- Por que BFS não substitui Dijkstra em um grafo ponderado?
- Como o algoritmo de Kahn revela uma dependência circular?
- O que um caminho aumentante modifica em um emparelhamento?
- Qual pergunta separa um problema euleriano de um hamiltoniano?
- Que casos extremos devem aparecer nos testes de uma implementação de grafo?

Modelo correto + representação adequada + algoritmo compatível + validação dos limites